Analysis of Training Configurations in Variational Autoencoder Design (including Activation Functions, Hyperparameters and more)

Release v1.0 (06/04/2026)

Introduction

A neural network, simply put, is a series of layers performing the same basic operation, that is multiplying inputs by learned weights and summing the results. If left unchecked, the entire network remains mathematically equivalent to a single layer regardless of the amount of layers you use, reducing its capability to only learning straight-line relationships, rendering it useless for complex applications such as recognising shapes in images, understanding languages etc.

Activation functions are the solution. When applied after each layer, they transform the value before it's passed on to the next layer, breaking the linearity and therefore giving it the capability to learn more complex and abstract patterns from data. However, the choice of activation function is just one of many decisions that determine how well a neural network learns and the quality of its output

This document examines the key components and configurable parameters that affect the behaviour of a Variational Autoencoder (VAE) built in TensorFlow and Keras, covering not just activation functions but optimisers, loss functions, hyperparameters (latent dimensionality, KL weighting, batch size etc.) as well as broader concepts such as architectural design choices, training callbacks, regularisation techniques and more. Acquiring a more extensive understanding around said topics gives us the ability for better design, training and finer tuning of these models.

Activation Functions

Different activation functions apply different transformation rules with each activation function having their own benefits and consequences which include how quickly the network learns, how well gradients flow backwards (although the end-goal of the project is to eliminate or significantly limit backpropagation), and the overall quality of the output produced.

ReLU (Rectified Linear Unit)

If the value received is negative, output 0. If positive, pass straight through.

activation='relu'

- Very fast to compute, works very well in practice
- Default choice for hidden layers in most modern networks
- However it can fall victim to a problem called "Dying ReLU", where many ReLU neurons stop learning, become inactive and output 0 for all inputs

Leaky ReLU

Fix for dying ReLU. Does not output exactly 0 for negatives, but a small fraction instead (e.g. 1/10 of input).

layers.LeakyReLU(alpha=0.1)

- Try when training seems stuck or loss plateaus early
- Slightly more expensive than ReLU with usually only a minor improvement

ELU (Exponential Linear Unit)

Uses a smooth exponential curve (as opposed to a flat line) for negatives, which can produce better-looking outputs.

activation='elu'

- Can converge faster than ReLU in some architectures
- Try when reconstructions look jagged or blocky

Sigmoid

(Currently used on the final output layer of the decoder only)
Squashes all input values into the range 0 to 1, where large negative numbers = closer to zero and zero = 0.5. Forms an S-shaped curve.

activation='sigmoid'

- Perfect for use with pixel values
- Do NOT use on hidden layers as it causes vanishing gradients

Tanh

Squashes input values to -1 to 1, like Sigmoid but centered around 0.

activation='tanh'

- Use on output layer if normalising pixel values to -1 to 1 instead of 0 to 1 (swap sigmoid for tanh)
- Can also help keep latent values bounded if used on z_mean
- Useful in Recurrent Neural Networks (RNNs, LSTMs) for text/sequence data

Softmax

Takes a list of numbers and converts them into probabilities that all add up to (e.g. [2.0, 1.0, 0.5] becomes [0.57, 0.29, 0.14], the biggest number gets the most probability).

activation='softmax'

- Only use for output-layers in multi-class classification only (inapplicable to VAEs)
- For when you need to pick one category from several
- Never used in hidden layers

ReLU, Leaky ReLU and ELU are for use on all hidden Conv/Dense layers. Sigmoid is for use when pixel values are normalised between 0 and 1 and on the final output layer. Tanh is also for the final output or latent layers, when normalised to -1 to 1. Start with ReLU as the default choice. Leaky ReLU will fix the "dying ReLU" issue. ELU may produce better outputs if current ones look too "blocky". Softmax is inapplicable to VAEs

---

Optimisers

An optimiser is the algorithm that updates the model's weights after each batch. It looks at the gradients (which direction each weight should move to reduce the loss) and decides how big of a step to take during the learning process.

Adam (Adaptive Moment Estimation)

The most popular optimiser in deep learning. Keeps track of both average gradient and how much the gradient has been varying, then adjusts step size per weight accordingly.

keras.optimizers.Adam(learning_rate=0.001, clipnorm=2.0)

- Default choice, works well on most problems without much interference or tuning
- Handles sparse gradients well
- Can sometimes find a solution that doesn't generalise as well as SGD

AdamW

Adam with weight decay (L2 regularisation / ridge regression included).

keras.optimizers.AdamW(learning_rate=0.001, weight_decay=0.01)

- Often produces slightly better generalisation than just Adam, a straightforward upgrade worth trying
- L2 regularisation reduces overfitting by adding a penalty term (weight decay). Due to squaring weights, it penalises large weights more heavily and forces a more even distribution
- Weight decay usually lies between 0.001 and 0.1

SGD (Stochastic Gradient Descent) with Momentum

The "classic" optimiser, where momentum carries it forward.

keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)

- Can sometimes find better, sharper solutions on well-tuned problems than Adam
- However, learning rate is fixed (but directional) and very sensitive
- Requires more experimentation to find optimal learning rate

RMSProp

Similar to Adam but tracks only the variance of gradients, not average.

keras.optimizers.RMS(learning_rate=0.001)

- Was the default before Adam
- Less commonly used for image models now

Adam should be used as a default starting point, then AdamW if overfitting is a problem. SGD with momentum can be used to squeeze out a little extra performance. RMSProp does not have much practical use.

Loss Functions

The loss function measures how wrong the model's output is. Training is the process of minimising this number. Binary Crossentropy, MSE and MAE are all reconstruction losses, unlike KL.

Binary Crossentropy

Measures how different the reconstructed image is from the original, pixel by pixel. Designed for values between 0 and 1.

keras.losses.binary_crossentropy(x, reconstruction)

- Good default when using sigmoid activation on output layer and grayscale inputs
- Tends to produce slightly blurry reconstructions due to averaging over pixels

Mean Squared Error (MSE)

Measures the average squared difference between original and reconstruction. Penalises large errors more heavily than small ones.

keras.losses.mse(x, reconstruction)

- The better choice for RGB inputs
- Try when binary crossentropy gives washed-out grey reconstructions
- Can produce slightly sharper edges
- When switching from Binary to MSE, change the output activation from sigmoid to linear (no activation)

Mean Absolute Error (MAE)

MSE without squaring. Every error is penalised proportionally regardless of size.

keras.losses.mean_absolute_error(x, reconstruction)

- Is more robust to outliers than MSE as a single wild outlier won't affect the loss as much
- Try if blurriness is an issue
- Tends to produce slightly sharper outputs as the penalties aren't as heavy as MSE
- However can be less stable to train

KL Divergence Loss

Mandatory mathematical requirement of a VAE. Forces latent space to be organised and smooth.

kl_loss = -0.5 * (z_log_var - z_mean² - exp(z_log_var) + 1)

- Lower KL loss means more organised latent space and better generation quality
- kl_weight specified when building/compiling the VAE, can be assigned to a constant

Categorical Cross-Entropy and Sparse Categorical Cross-Entropy are inapplicable to VAEs

---

Key Hyperparameters

The parameters assigned to constants at the start of the code for easier tuning. These parameters have a big impact on how well the model trains and on the quality of results.

LATENT_DIM

The size of the bottleneck. How many numbers the encoder compresses each image down to.

```
def build_encoder(latent_dim=LATENT_DIM)
```

Typical range: 2 - 256 (more complex datasets require higher values)

- If set too small, the model can't capture enough information and reconstructions are blurry
- If too large, latent space becomes unorganised and generation quality suffers
- Setting value to 2 is useful for visualisation as you can plot the whole latent space in 2D
- 8 - 16 is a good range for a simple database such as MNIST
- Complex datasets need much higher values (e.g. CIFAR-10 needs 32 - 64, high resolution faces can require much more like 256)

KL_WEIGHT

The multiplier on the KL loss. Controls the balance between the reconstruction and KL loss.

```
vae = VAE(encoder, decoder, kl_weight=KL_WEIGHT, name='vae')
```

Typical range: 0.1 - 50.0 (start from 1 and adjust from there)

- Higher values result in a more organised latent space, better generation and blurrier reconstructions
- Lower values result in sharper reconstructions but disorganised latent space, generation will be more noisy
- If generations are too noisy, increase value
- If reconstructions are too blurry, decrease value

BATCH_SIZE

The amount of images the model sees before updating its weights once.

```
train_dataset = train_dataset.shuffle(10000).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

Typical range: Depends on resources available and size of images (lower for bigger images or if less VRAM available)

- Higher values result in more stable gradients, can use higher learning rates but use more GPU memory
- Lower values result in noisier gradients, but the noise can help escape local minima

LEARNING_RATE

The size of the step the optimiser takes when updating weights (most important hyperparameter)

```
tvae.compile(optimizer=keras.optimizers.Adam(
learning_rate=LEARNING_RATE,
clipnorm=CLIPNORM))
```

Typical range: 0.001 is the standard Adam default (try lower for more complex datasets)

- If too high, the loss bounces around or increases/decreases by far too much
- If too low, the training is very slow and it may get stuck

CLIPNORM

Safety mechanism against outliers or out of control gradients. If any gradient exceeds this value, scale it down.

tvae.compile(optimizer=keras.optimizers.Adam(
learning_rate=LEARNING_RATE,
clipnorm=CLIPNORM))

Typical range: Variable, in our case 1.0 - 5.0 (set to None to disable)

- If training crashes with NaN losses, lower the value
- If training looks too conservative, raise or set to None

Architecture Choices

Changes to the encoder/decoder structure

Number of Filters in Conv2D Layers

Each filter is a pattern detector.
(Assigned to constant BASE_FILTERS)

BASE_FILTERS = 32
layers.Conv2D(BASE_FILTERS, …, …) (32 filters to start off with)

- More filters result in higher quality representations, but are slower and use more GPU memory
- Fewer filters are faster and lighter but may miss details
- Doubling pattern used the more layers are added (e.g. first layer 32, then 64, then 128 and so on)

Kernel Size

The size of the sliding window in Conv2D (3 means a 3×3 pixel window)

layers.Conv2D(..., 3, …) (3x3 pixel window)

- Larger kernels see a bigger area at once but are slower
- 3 is the default/standard for most image networks

Number of Layers

(Assigned to constant NUM_LAYERS)

- Fewer (e.g. 2) layers will train faster, sufficient for simple datasets
- More (4+) are needed for larger or more complex images

Dropout

Randomly switches off neurons during training to reduce overfitting.

layers.Dropout(0.2) (switch 20% off randomly)
Typical range: 0.1 - 0.5

- Add after Dense or Conv layers if the training loss is much lower than validation loss
- Do not add dropout in the decoder's final layers as it makes reconstructions noisy

---

Regularisation

Techniques to prevent the model from memorising training data.

Batch Normalisation

Rescales layer outputs to have a consistent mean and spread.

layers.BatchNormalization()

- Stabilises training, very impactful addition to modern networks.
- Use before Dropout if both are used

L2 Regularisation (weight decay)

Adds a penalty to the loss for having large weights.

layers.Dense(64, kernel_regularizer=keras.regularizers.l2(0.01))

Typical range: 0.0001 - 0.01

- Add to Dense layers if overfitting is occurring in the model
- Can just use AdamW which has this feature built-in

Training Callbacks

Callbacks run automatically at the end of each epoch and modify training.

EarlyStopping

EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

- patience: amount of epochs without improvement before stopping. Lower stops sooner
- restore_best_weights rolls back to the best checkpoint, is usually set to True
- min_delta: minimum change that counts as improvement, ignores tiny fluctuations

ReduceLROnPlateau

ReduceLROnPlateau(monitor='val_loss', factor=0.75, patience=5, min_lr=1e-6)

- factor: multiplier when reducing learning rate. 0.75 is gentle, 0.5 is aggressive
- patience: epochs without improvement before reducing
- min_lr: floor on the learning rate (1e-6 = 0.000001)

ModelCheckpoint

Saves model weights to disk every time validation loss improves. Useful for long training runs.

keras.callbacks.ModelCheckpoint('best_model.keras', save_best_only=True)

Data Pipeline

How data is fed into the model. Affects training quality and speed.

Shuffle Buffer

How many images to load into memory for random shuffling. Larger = more random.

train_dataset.shuffle(10000)

- For 60,000 training images, a buffer of 10,000–60,000 is typical
- Setting it to 60,000 gives maximum randomness

Prefetch

Loads the next batch in the background while the GPU processes the current one for better speed/efficiency.

train_dataset.shuffle(10000).prefetch(tf.data.AUTOTUNE)

- AUTOTUNE lets the model decide what's best for the resources available, recommended to leave as-is

---

Order of Tuning

The recommended order of where to start experimenting for different results is as follows:

- The most impactful constants first: LATENT_DIM, KL_WEIGHT
- Then learning rate (lower if training is unstable, raise if training is too slow)
- Architecture changes such as adding/removing a conv layer, doubling/halving filter counts etc. (will have the biggest impact on training time)
- As a "last resort" in a sense you can (discouraged as MSE with tanh is widely considered the most stable and default choice for colour images)
    - Try a different reconstruction loss (e.g. swap Binary Crossentropy for MSE, remembering to change output activation from sigmoid to linear if necessary)
    - Try different activation functions based on reconstruction quality

It's best practice to change one at a time and note the difference. It becomes easier to lose track of what causes improvements if multiple changes are applied at once. Having these assigned to constants at the start of your code can make tuning a lot less of a hassle too.

Understanding Metrics

How to interpret the cell outputs and graphs which are outputted during training

Kl_loss

KL loss is essentially the sum of how far each latent dimension's distribution is from a standard normal.
A rough rule of thumb is that a healthy KL loss tends to land somewhere in the range of 0.05–0.2 per latent dimension when the model is learning well (e.g. LATENT_DIM=64 × (0.05 to 0.2) = 3 to 13).
KL weight has a direct correlation with it. The lower KL weight is the lower KL loss is, so if loss is too low raise the weight until the loss lands in a range that gives you a suitable balance between a good reconstruction/generation.

Training loss noise

Training loss should decrease smoothly and not bounce around. Batch size can be increased to remedy this. The graph plotting validation loss and training loss over epochs which should be drawn and shown as part of your last cell output is a very useful resource to see how noisy the progression is. Ideally it should be a smooth curve.

Relationship between validation and training loss

Validation and training loss should decrease together and track closely. The graph outputted at the end again is very useful for determining whether this is true or not. There should not be too big of a gap between the two values either. If they diverge instead of converge over epochs, then overfitting is present.

Validation loss plateau

If the reduction in validation loss plateaus or flattens over 15-20+ epochs, then the model has hit a ceiling/wall and has run out of capacity to learn or train any further. You want to see validation loss still slowly improving at the end of training until you have your desired quality of reconstruction/generation output.

Reaching model capacity and improvements

If you see all the correct metrics (training and validation loss decreases smoothly and together with a small gap in between each other, a plateau before the training finishes, the reductions in learning rate helping but not breaking through the plateau) then the model has hit a ceiling and can no longer improve without expanding its capacity.

Ways to increase model capacity include:

- Increasing the amount of filters (adjusting the BASE_FILTERS constant)
- Increasing latent dimensions (LATENT_DIM)
- Upscaling the image first to allow for more layers (NUM_LAYERS) and therefore a larger bottleneck size (BOTTLENECK_DIM)
    - As BOTTLENECK_DIM ^ (NUM_LAYERS + 1) must equal IMAGE_SIZE (e.g. 2^(4+1) = 32, image size halves by 2 each layer due to the stride length being 2 (32 -> 16 -> 8 -> 4 -> 2))
    - A larger bottleneck dimension allows more information to get through and is less harsh on the information it deems obsolete and filters out

---

Other Notes

Different Image Sizes

If your dataset has different image sizes, use this first to resize it all to a fixed target size.

```
tf.image.resize(image, [target_height, target_width])
```

Data Normalisation

MNIST (a grayscale database) divides by 255 to get 0 – 1 and uses Binary and sigmoid.
For RGB, it is recommended to use MSE with tanh instead of Binary and sigmoid and normalise to -1 to 1:

```
outputs = layers.Conv2D(3, 3, padding='same', activation='tanh')
x = (x / 127.5) - 1.0
```

You can remove the activation and let it default to linear, which is a mathematically pure pairing with MSE but it can be unstable early in training and relies on ClipNorm and good initialisation to stay under control.
MSE with tanh is much more stable in training.

Data Augmentation

For small or complex datasets, augmentation artificially increases variety by applying random transformations during training. This reduces overfitting.

```
x = tf.image.random_flip_left_right(x)
x = tf.image.random_brightness(x, max_delta=0.1)
x = tf.image.random_contrast(x, lower=0.9, upper=1.1)
```

- Only apply augmentation to training data, never validation data
- Flipping is almost always safe to add for natural images
- Be careful with colour jitter